Blackboard Exercise 2: Worked solutions

Algorithms, Programming, and Data Representation · Module 92313 · Blackboard Exercise 2

From Fundamentals to AI Apocalypse · personal study guide

Contents

Exercise 2.1: Natural numbers 2

Exercise 2.2: Floating-point numbers 3

Exercise 2.3: Signed and unsigned integers 3

Exercise 2.4: Text representation and interpretation 4

Exercise 2.5: Boolean algebra 4

Answers 5

Made by Thanate Amornjiraporn (Focus)

A worked route through number bases, binary fractions, integer encodings, ASCII and Boolean algebra. Try each subpart before opening its steps.

Exercise 2.1: Natural numbers

Connected topic: Number systems and data representation (§3.1–3.3)

2.1.1 Number representations

Try first: Complete the five conversions; show how the first and last binary rows convert directly to octal and hexadecimal.

Answers: see "Exercise 2.1: Natural numbers" under Answers at the end.

2.1.2 Range of an unsigned representation

Try first: Find the ranges for 8, 16, and 32 bits, derive the general rule, then find the fewest bits that cover 0 through 1000.

Answers: see "Exercise 2.1: Natural numbers" under Answers at the end.

2.1.3 Representation and storage

Try first: Decide which listed values fit an unsigned byte, identify equal values, and explain why distinct spellings can denote the same number.

Answers: see "Exercise 2.1: Natural numbers" under Answers at the end.

2.1.4 Arithmetic

Try first: Calculate the four sums, difference, and products in the operand bases without converting the operands to decimal first; decide whether each result fits an unsigned byte.

Answers: see "Exercise 2.1: Natural numbers" under Answers at the end.

2.1.5 Overflow

Try first: For each decimal addition, find the mathematical sum, whether an 8-bit unsigned overflow occurs, and the stored byte. Then explain the NumPy result for `uint8(250) + uint8(13)`.

Answers: see "Exercise 2.1: Natural numbers" under Answers at the end.

2.1.6 Working with an unknown radix

Try first: The numeral 243 in base r has decimal value 99. Find r and decide whether 354_r and 358_r are valid.

Answers: see "Exercise 2.1: Natural numbers" under Answers at the end.

Made by Thanate Amornjirapong (Focus)

Exercise 2.2: Floating-point numbers

Connected topic: Binary fractions and floating-point representation (§4.1)

2.2.1 Binary fractions

Try first: Complete the four binary/decimal conversions and show one calculation in each direction.

Answers: see "Exercise 2.2: Floating-point numbers" under Answers at the end.

2.2.2 Finite and repeating representations

Try first: Convert 0.1_{10} until its repeating pattern is visible. Decide which listed fractions terminate in binary, then state the general condition.

Answers: see "Exercise 2.2: Floating-point numbers" under Answers at the end.

Exercise 2.3: Signed and unsigned integers

Connected topic: Integer representations and arithmetic (§3.4)

2.3.1 Two's complement

Try first: Complete the table for the three bit patterns and the two decimal values, using exactly eight bits.

Answers: see "Exercise 2.3: Signed and unsigned integers" under Answers at the end.

2.3.2 Range and interpretation

Try first: Find the 8-bit and general n-bit ranges, explain the asymmetric counts, and interpret 11110110_2 both ways.

Answers: see "Exercise 2.3: Signed and unsigned integers" under Answers at the end.

2.3.3 Signed overflow

Try first: Add 01100101_2 and 00111110_2, interpret all three patterns as signed decimal, and test for signed overflow.

Answers: see "Exercise 2.3: Signed and unsigned integers" under Answers at the end.

Made by Thanate Angornjiraporn (Focus)

Exercise 2.4: Text representation and interpretation

Connected topic: Text and character representation (§3.5)

2.4.1 ASCII

Try first: Complete the table and encode AIBE as four 8-bit ASCII bytes.

Answers: see "Exercise 2.4: Text representation and interpretation" under Answers at the end.

2.4.2 One bit pattern, different meanings

Try first: Interpret 01000001_2 as unsigned, as 8-bit two's complement, and as ASCII. State what extra information a reader needs.

Answers: see "Exercise 2.4: Text representation and interpretation" under Answers at the end.

Exercise 2.5: Boolean algebra

Connected topic: Boolean algebra identities (§1.1–1.2)

2.5.1 Truth table

Try first: Fill the columns A', $A \cdot B$, $A + B$ and $A + (B \cdot C)$ for all input rows in the given order.

Answers: see "Exercise 2.5: Boolean algebra" under Answers at the end.

2.5.2 Simplification

Try first: Simplify the four expressions and name the Boolean law used at each step.

Answers: see "Exercise 2.5: Boolean algebra" under Answers at the end.

2.5.3 Equivalence and construction

Try first: Compare $A + (B \cdot C)$ with $(A + B)(A + C)$ using algebra and a truth table; construct $F = 1$ exactly when at least two inputs are 1.

Answers: see "Exercise 2.5: Boolean algebra" under Answers at the end.

Made by Thanate Amornjiraporn (Focus)

Answers

Exercise 2.1: Natural numbers

2.1.1 Number representations · worked solution

1. Read each subscript as the base. To go from binary to octal, group the bits in threes from the right; to hexadecimal, in fours. Pad with zeros on the left only, where a group is short.
2. The first and last rows, converted directly:

$$10110110_2 = 010\,110\,110_2 = 266_8$$

$$10110110_2 = 1011\,0110_2 = \mathrm{B6}_{16}$$

$$110101011001_2 = 110\,101\,011\,001_2 = 6531_8$$

$$110101011001_2 = 1101\,0101\,1001_2 = \mathrm{D59}_{16}$$

3. The completed table, with the given entry of each row in bold:

Binary	Octal	Decimal	Hexadecimal
10110110	266	182	B6
11100111	**347**	231	E7
10011100	234	**156**	9C
1011010111	1327	727	**2D7**
110101011001	6531	3417	D59

4. Grouping works because $8 = 2^3$ and $16 = 2^4$: one octal digit stands for exactly three bits and one hexadecimal digit for exactly four, so each group converts on its own and nothing carries from one group to the next.
5. **Check:** expand one row back in its own base, for example $\mathrm{B6}_{16} = 11 \cdot 16 + 6 = 182$ and $347_8 = 3 \cdot 64 + 4 \cdot 8 + 7 = 231$.

Ma

Exercise 2.1: Natural numbers

2.1.2 Range of an unsigned representation · worked solution

1. An n-bit unsigned pattern has place values 2^0 to 2^{n-1}. The smallest value is all zeros; the largest is all ones, the sum of every place value.

2.

Bits	Smallest	Largest
8	0	$2^8 - 1 = 255$
16	0	$2^{16} - 1 = 65\,535$
32	0	$2^{32} - 1 = 4\,294\,967\,295$

In general an n-bit unsigned integer covers

$$0 \leq x \leq 2^n - 1$$

3. The numbers 0 to 1000 are 1001 different values, and n bits give 2^n patterns, so we need the smallest n with $2^n \geq 1001$:

$$2^9 = 512 < 1001 \leq 1024 = 2^{10}$$

10 bits are required; 9 bits stop at 511.

Exercise 2.1: Natural numbers

2.1.3 Representation and storage · worked solution

1. An unsigned 8-bit integer stores the values 0 to 255. Work out each value, then compare it with that range.
2. Count the digits of the second binary value: 100000000_2 has **nine** digits, a 1 followed by eight zeros, so it is 2^8.

$$11111111_2 = 255$$

$$100000000_2 = 2^8 = 256$$

$$\mathrm{FF}_{16} = 15 \cdot 16 + 15 = 255$$

$$256_{10} = 256$$

3. **Fit in an unsigned byte:** 11111111_2 and FF_{16}. **Do not fit:** 100000000_2 and 256_{10}; 256 needs nine bits.
4. **Same value, different spelling:** $11111111_2 = \mathrm{FF}_{16} = 255$, and $100000000_2 = 256_{10} = 256$. A numeral is a weighted sum of its digits in its base; different bases give different digit strings for the same number. A byte holds the value's bit pattern, not the way it was written.

Exercise 2.1: Natural numbers

2.1.4 Arithmetic · worked solution

1. Keep the base fixed throughout: line the digits up by place value and carry or borrow in that base, as in decimal written arithmetic.
2. **a)** Add column by column from the right; a column that sums to 2 or 3 writes 0 or 1 and carries 1 to the left.

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
carry		1	1	1	1			
	1	0	1	1	0	1	1	0
+	0	0	1	0	1	1	0	1
=	1	1	1	0	0	0	1	1

   The sum is 11100011_2 (227).
3. **b)** Subtract from the right; where the top digit is too small, borrow 1 from the column to the left (it is worth 2 here). The marked columns lent a 1.

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
lent		1	1		1	1	1	
	1	1	0	0	1	0	1	0
−	0	0	1	1	0	1	1	1
=	1	0	0	1	0	0	1	1

   The difference is 10010011_2 (147).
4. **c)** $00000100_2 = 2^2$, so multiplying by it shifts every bit two places left: $00101101_2 \times 00000100_2 = 10110100_2$ (180). The two bits pushed out at the top were 0, so nothing is lost.
5. **d)** In hexadecimal, 7 + C = 19 = 16 + 3: write 3 and carry 1. Then A + 3 + 1 = 14 = E. So $A7_{16} + 3C_{16} = E3_{16}$ (227).
6. **Fit:** all four results are at most $FF_{16} = 255$, and the difference is not negative, so each fits an unsigned byte. **Check:** a) and d) are the same sum, 182 + 45 = 167 + 60 = 227.

Made

Exercise 2.1: Natural numbers

2.1.5 Overflow · worked solution

1. An unsigned byte keeps only the low eight bits of a result, which is the result modulo 256. Overflow happens when the true sum is larger than 255.

2.

Addition	Mathematical result	Overflow?	Stored in 8 bits
$200 + 55$	255	no	255
$200 + 56$	256	yes	$256 - 256 = 0$
$250 + 13$	263	yes	$263 - 256 = 7$
$128 + 128$	256	yes	0

3. **Condition:** two unsigned bytes a and b overflow exactly when

$$a + b > 255$$

which is the same as a carry out of the leftmost bit; the stored value is then $a + b - 256$.

4. **The NumPy lines:** both operands are `uint8`, so the sum is also computed as a `uint8`: $250 + 13 = 263$ wraps to $263 - 256 = 7$, and `print` shows `7`. Recent NumPy versions also warn about an overflow in the scalar addition. Python's own `250 + 13` would give 263, because Python integers have no fixed width.

5. **Check:** $200 + 55$ sits exactly on the limit 255, so it does not overflow.

Exercise 2.1: Natural numbers

2.1.6 Working with an unknown radix · worked solution

1. Expand by place value; a radix must be a whole number larger than every digit used, so $r \geq 5$ here.

$$243_r = 2r^2 + 4r + 3 = 99$$

2. Rearrange and factorise:

$$2r^2 + 4r - 96 = 0$$

$$r^2 + 2r - 48 = 0$$

$$(r + 8)(r - 6) = 0$$

A radix is positive, so $\boldsymbol{r = 6}$; -8 is rejected.

3. In base six the digits are 0 to 5. 354_6 is **valid**: every digit is below 6. 358_6 is **not valid**: 8 is not a base-six digit.

4. **Check:** $2 \cdot 36 + 4 \cdot 6 + 3 = 72 + 24 + 3 = 99$.

Exercise 2.2: Floating-point numbers

2.2.1 Binary fractions · worked solution

1. After the binary point the places are worth $2^{-1} = \frac{1}{2}$, $2^{-2} = \frac{1}{4}$, $2^{-3} = \frac{1}{8}$ and so on. Binary to decimal: add the place values of the 1 bits. Decimal to binary: double the fraction again and again; each whole part that appears is the next bit.
2. **Binary to decimal:**

$$0.1011_2 = \frac{1}{2} + \frac{1}{8} + \frac{1}{16} = 0.6875$$

3. **Decimal to binary,** for 0.625:

Fraction	× 2	Bit
0.625	1.25	1
0.25	0.5	0
0.5	1.0	1

The fraction reaches 0, so $0.625_{10} = 0.101_2$. For 5.375, the whole part $5 = 101_2$ and 0.375 doubles to 0.75 (0), 1.5 (1), 1.0 (1), so $5.375_{10} = 101.011_2$.

4. The completed table:

Binary	Decimal
0.1011	0.6875
10.011	2.375
0.101	**0.625**
101.011	**5.375**

5. **Check:** convert back, $101.011_2 = 4 + 1 + \frac{1}{4} + \frac{1}{8} = 5.375$.

Made by

Exercise 2.2: Floating-point numbers

2.2.2 Finite and repeating representations · worked solution

1. Use the doubling method from 2.2.1 on 0.1 and watch the fractions that remain:

Fraction	× 2	Bit
0.1	0.2	0
0.2	0.4	0
0.4	0.8	0
0.8	1.6	1
0.6	1.2	1
0.2	0.4	0
…	…	…

2. The fraction 0.2 has come back, so from there the bits 0011 repeat for ever:

$$0.1_{10} = 0.0001\,1001\,1001\,1\ldots_2$$

It never reaches 0 because $\frac{1}{10}$ has the factor 5 in its denominator, and doubling can never cancel a 5.

3. Check each fraction in lowest terms: $\frac{1}{2} = 0.1_2$, $\frac{3}{8} = 0.011_2$ and $\frac{5}{16} = 0.0101_2$ are **finite**; $\frac{1}{10}$ and $\frac{7}{20}$ (denominators $2 \cdot 5$ and $4 \cdot 5$) **repeat**.

4. **Condition:** a fraction $\frac{p}{q}$ in lowest terms has a finite binary expansion exactly when q is a power of 2, that is $q = 2^k$. Then multiplying by 2^k gives a whole number, which ends the doubling after k bits.

Made by Thana

Exercise 2.3: Signed and unsigned integers

2.3.1 Two's complement · worked solution

1. In 8-bit two's complement the leftmost bit is worth -128 and the others keep their usual positive weights. To encode a negative number, write its magnitude in 8 bits, invert every bit and add 1.
2. For -37: $37 = 00100101_2$; inverted 11011010_2; plus 1 gives 11011011_2. For -8: $8 = 00001000_2 \to 11110111_2 \to 11111000_2$.
3. The completed table:

Binary	Decimal
01111111	127
11111111	-1
10000000	-128
11111000	$\mathbf{-8}$
11011011	$\mathbf{-37}$

4. **Check** with the -128 weight: $11111000_2 = -128 + 64 + 32 + 16 + 8 = -8$ and $11011011_2 = -128 + 64 + 16 + 8 + 2 + 1 = -37$.

Exercise 2.3: Signed and unsigned integers

2.3.2 Range and interpretation · worked solution

1. In n-bit two's complement the leading bit is worth -2^{n-1} and the other bits add up to at most $2^{n-1} - 1$.
2. For 8 bits the range is -128 to 127; in general

$$-2^{n-1} \le x \le 2^{n-1} - 1$$

   There are 2^n patterns. Half of them, 2^{n-1}, start with 1 and are negative; the other half start with 0, and one of those is zero, which leaves $2^{n-1} - 1$ positive values. So there is one more negative value than positive.
3. 11110110_2 read as unsigned is $128 + 64 + 32 + 16 + 4 + 2 = \mathbf{246}$; read as two's complement the first bit counts -128 instead, giving $246 - 256 = \mathbf{-10}$.
4. The bits themselves carry no sign. The type the program declares decides whether the leading bit is worth $+128$ or -128.

Exercise 2.3: Signed and unsigned integers

2.3.3 Signed overflow · worked solution

1. Add the patterns as unsigned bits, then read the operands and the result as signed values.

$$01100101_2 + 00111110_2 = 10100011_2$$

2. Both operands start with 0, so they are positive: +101 and +62. Their true sum 163 is above 127. The result starts with 1 and reads as $-128 + 32 + 2 + 1 = -93$, which is $163 - 256$.

3. **Signed overflow occurs.** Two positive operands gave a negative result. In general an addition overflows exactly when both operands have the same sign and the result's sign differs; adding a positive and a negative number can never overflow.

Exercise 2.4: Text representation and interpretation

2.4.1 ASCII · worked solution

1. ASCII gives each character a code from 0 to 127, stored in one byte. Convert the binary to decimal and look the code up, or the other way round.

Binary	Decimal	ASCII
01000001	65	A
01100001	97	a
00110000	**48**	0 (the digit)

2.

3. **AIBE** in 8-bit ASCII, one byte per letter:

Character	Hexadecimal	Binary
A	41	01000001
I	49	01001001
B	42	01000010
E	45	01000101

4. **Check:** a capital and a small letter differ by 32, a single bit (01000001_2 and 01100001_2), and code 48 is the character '0', not the number zero.

Exercise 2.4: Text representation and interpretation

2.4.2 One bit pattern, different meanings · worked solution

1. Evaluate the same eight bits under each encoding rule rather than changing the bits.

2. Unsigned interpretation: 65. Two's-complement interpretation: 65, because the leading bit is zero. ASCII interpretation: 'A'.

3. The data's type or agreed encoding supplies the meaning: for example, unsigned byte, signed 8-bit integer, or ASCII character. Bits alone do not specify their interpretation.

Exercise 2.5: Boolean algebra

2.5.1 Truth table · worked solution

1. Evaluate each row on its own: $'$ flips a bit, $\cdot$ is AND, $+$ is OR, and the brackets make $B \cdot C$ happen before the outer OR.
2.

A	B	C	A'	$A \cdot B$	$A + B$	$A + (B \cdot C)$
0	0	0	1	0	0	0
0	0	1	1	0	0	0
0	1	0	1	0	1	0
0	1	1	1	0	1	1
1	0	0	0	0	1	1
1	0	1	0	0	1	1
1	1	0	0	1	1	1
1	1	1	0	1	1	1

3. **Check** row 011: $A = 0$ but $B \cdot C = 1$, so the last column is 1. In every row with $A = 1$ both OR columns are 1.

Exercise 2.5: Boolean algebra

2.5.2 Simplification · worked solution

1. Rewrite one law at a time and name it, so each step can be checked.
2. $$A + A' = 1$$

   $$A \cdot A' = 0$$

   both by the complement law.
3. $$A + (A' \cdot B) = (A + A') \cdot (A + B)$$

   $$= 1 \cdot (A + B) = A + B$$

   by distributivity of $+$ over $\cdot$, then the complement and identity laws.
4. $$(A + B)(A + B') = A + B \cdot B'$$

   $$= A + 0 = A$$

   by distributivity, then the complement and identity laws.
5. **Check** the last two by putting $A = 0$ and $A = 1$: the left sides then give B and 1, and 0 and 1, matching $A + B$ and A.

Exercise 2.5: Boolean algebra

2.5.3 Equivalence and construction · worked solution

1. **Equivalence by algebra:** multiply out the product, then use idempotence ($A \cdot A = A$) and absorption ($A + A \cdot X = A$):

$$(A + B)(A + C) = A \cdot A + A \cdot C + A \cdot B + B \cdot C$$

$$= A + A \cdot C + A \cdot B + B \cdot C = A + B \cdot C$$

2. **By truth table:** both expressions give 0, 0, 0, 1, 1, 1, 1, 1 for the rows 000 to 111 (the last column of 2.5.1), so they are equivalent.

3. **At least two inputs are 1:**

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

   Write one product term for each row where $F = 1$:

$$F = A'BC + AB'C + ABC' + ABC$$

4. Copy the term ABC twice more (idempotence) and pair it with each other term; each pair loses the variable that differs:

$$A'BC + ABC = BC$$

$$AB'C + ABC = AC$$

$$ABC' + ABC = AB$$

$$F = AB + AC + BC$$

5. **Check:** with exactly two inputs 1, one pair term is 1; with all three, all three terms are; with one or none, no pair is complete.

Personal worked solutions based on the APDR Blackboard Exercise 2 sheet, written in my own words; no exercise-sheet prose or figures are reproduced. Not an official FAU publication; check course details against the current materials. Made by **Thanate Amornjiraporn (Focus)**.